Fellow

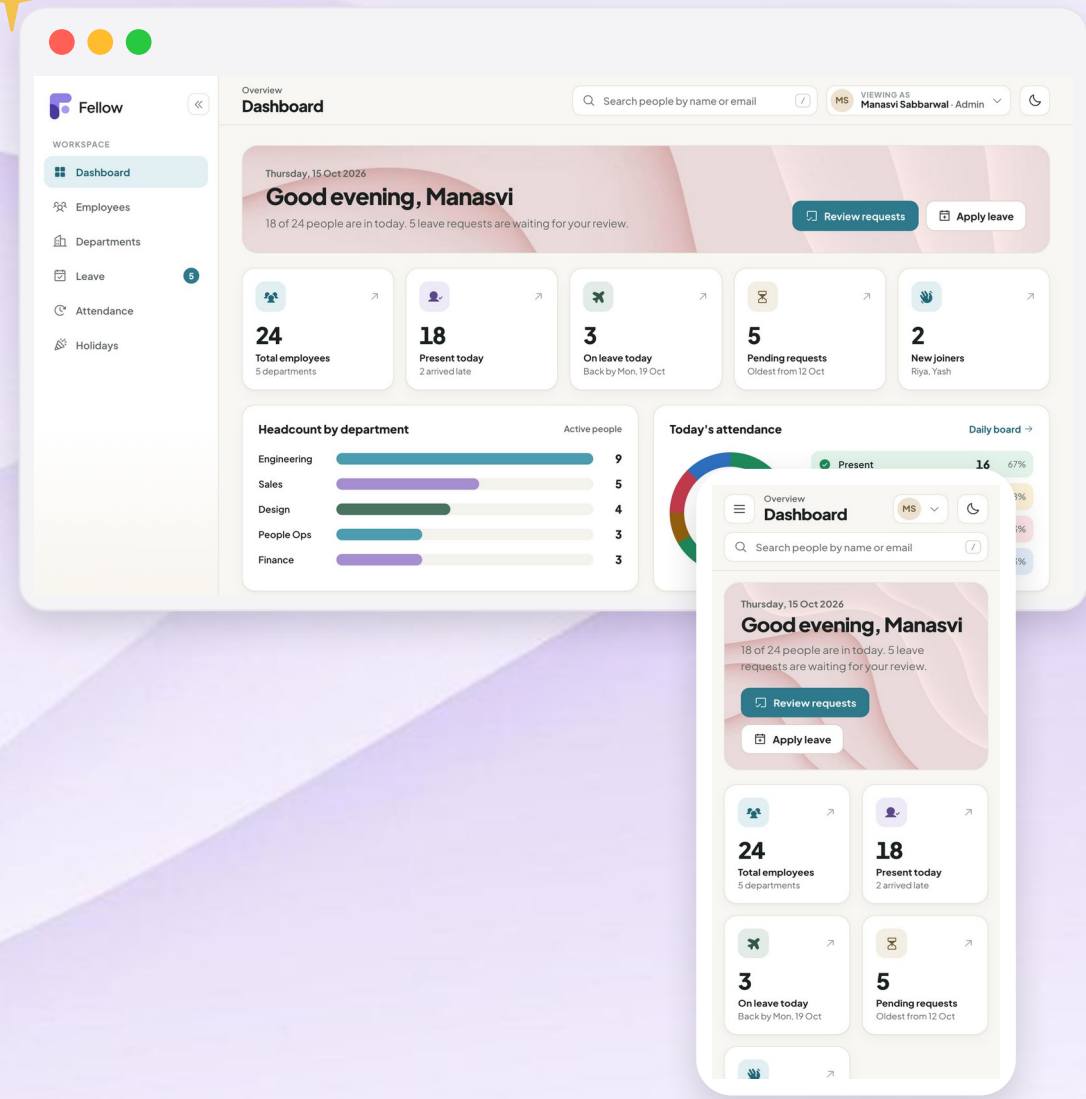
an HR app, shipped the DevOps way

From a React screen to a monitored Kubernetes deployment: tested, scanned, containerised, provisioned with Terraform and delivered through GitOps.

DEVOPS CAPSTONE

OCT 2026

Manasvi Sabbarwal



THE PROJECT

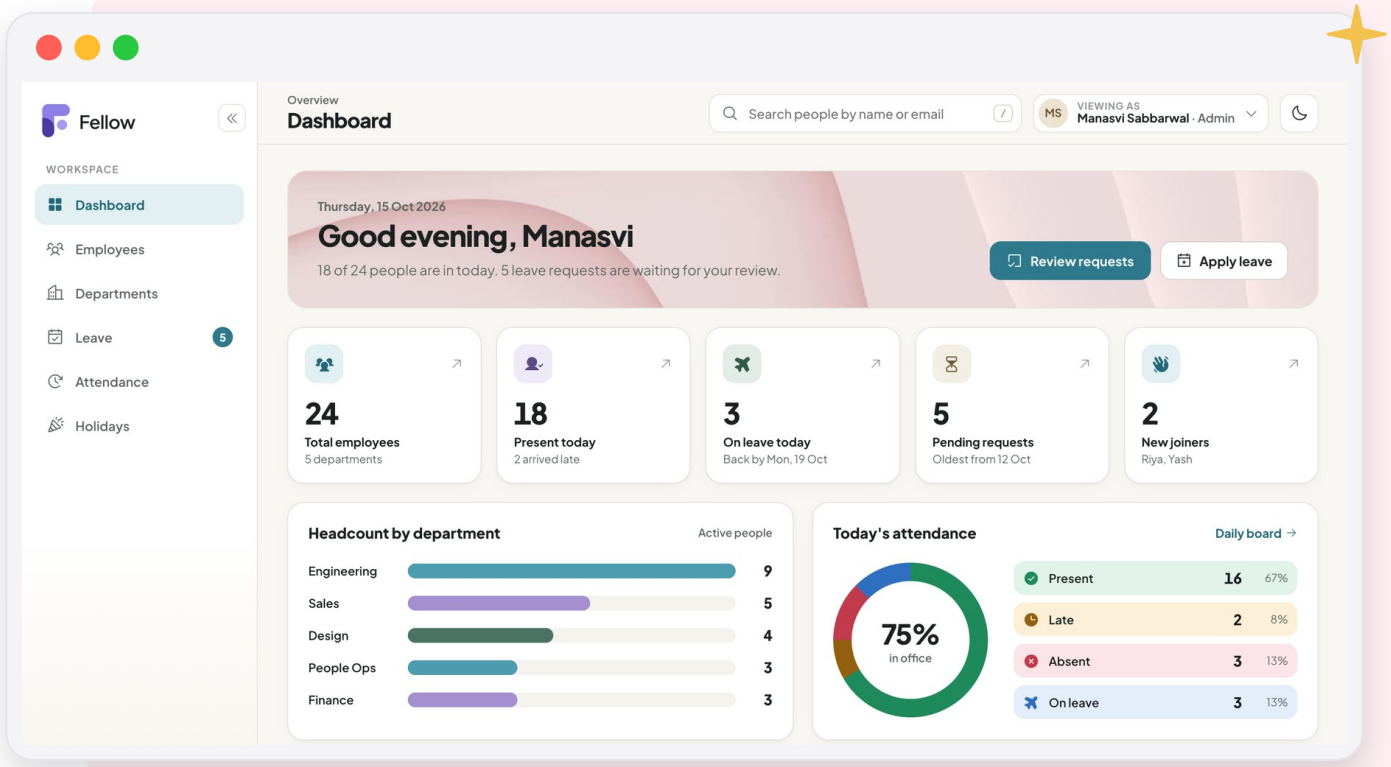
Meet Fellow

A lightweight HR workspace for a 20 to 200 person company. People, departments, leave and attendance live in one place, with a dashboard that answers "who is in today?" at a glance.

26 REST endpoints

7 screens, light and dark

34 automated tests



Live on staging.fellow.localhost

FEATURES

Everything HR needs, nothing it doesn't

People directory

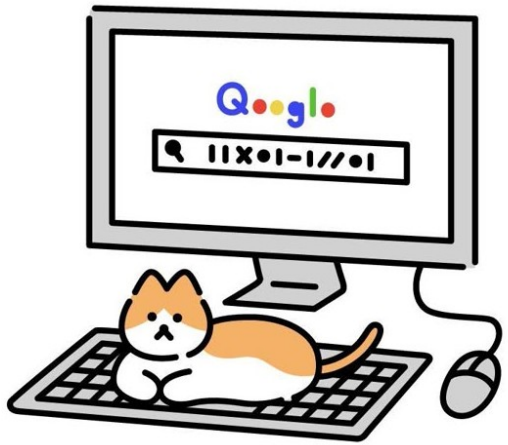
Search, filter, table or card view, full profiles with tenure and team.

Departments

Heads, live headcount, delete blocked while people remain.

Leave

Balances per type, weekends and holidays skipped, approve or reject.



busy cat, busy HR team

Attendance

Server-timed check in and out, late after 09:30, daily board.

Holidays

Public and optional days that feed leave and attendance rules.

Dashboard and roles

"Viewing as" switcher: admins approve, employees self-serve.

The stack

Build

- React 18 + Vite
- react-router
- FastAPI
- PostgreSQL 16
- SQLAlchemy
- Alembic

Verify

- pytest (34)
- Trivy
- actionlint
- helm lint
- Code review

Ship

- Docker (non-root)
- GitHub Actions
- GHCR, SHA tags
- Argo CD

Run

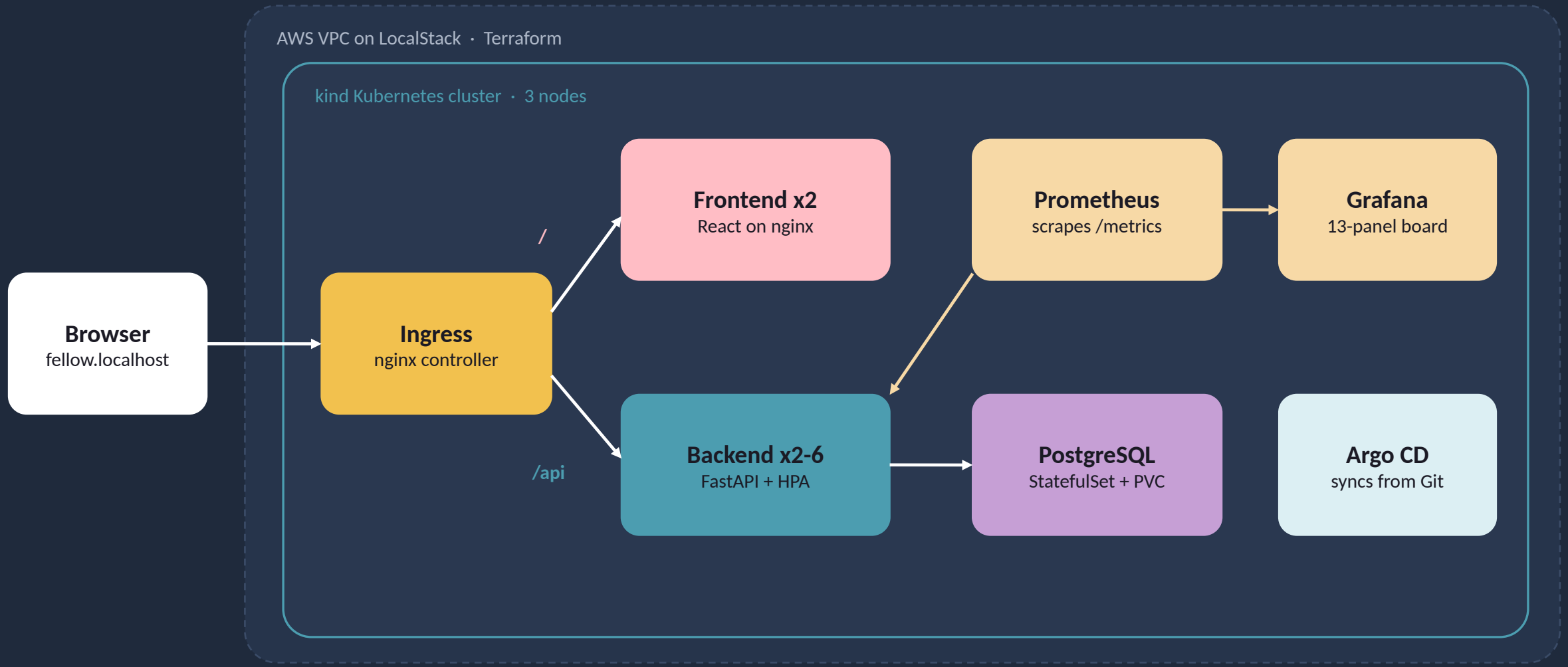
- Terraform
- AWS EKS
- LocalStack
- kind
- Helm
- HPA

Observe

- Prometheus
- Grafana
- ServiceMonitor
- kube-state-metrics

ARCHITECTURE

How a request travels



Push once, ship safely

1. **Push** — git push to main
2. **Test** — pytest + frontend build
3. **Build** — two Docker images
4. **Scan** — Trivy, fail on HIGH
5. **Publish** — GHCR, tag = commit SHA
6. **Promote** — bot commits the new tag
7. **Sync** — Argo CD rolls staging

5 / 5

jobs green on every push to main



~2 min

from commit to a pod running it

INFRASTRUCTURE AS CODE

Terraform, end to end

What terraform apply creates on AWS

VPC	10.20.0.0/16 in ap-southeast-2
Subnets	2 public + 2 private, 2 AZs
Routing	internet gateway, NAT, route tables
EKS	1.35, 2 x m7i-flex.large nodes
Add-ons	Pod Identity, EBS CSI driver



reviewing terraform plan

26 applied on real AWS

15 locally, LocalStack + kind

2 AZs, one NAT gateway

terraform plan **terraform apply** **terraform destroy**

Two workspaces: LocalStack + kind for free, real EKS in Sydney.

One chart, kind and EKS

Helm chart

Deployments, Services, Ingress, HPA, Postgres StatefulSet, ServiceMonitor and a Grafana dashboard.

Safe migrations

An init container runs Alembic under a Postgres advisory lock, so replicas never race.

Locked down

Non-root pods, dropped capabilities, seccomp, secrets kept out of Git.

kind and EKS

helm upgrade for dev; Argo CD syncs staging on kind and on real EKS from Git.

Load test

2 → 6

backend pods, scaled by the HPA when CPU hit 350% of target

fellow.localhost

EKS via AWS NLB

OBSERVABILITY

You can’t fix what you can’t see

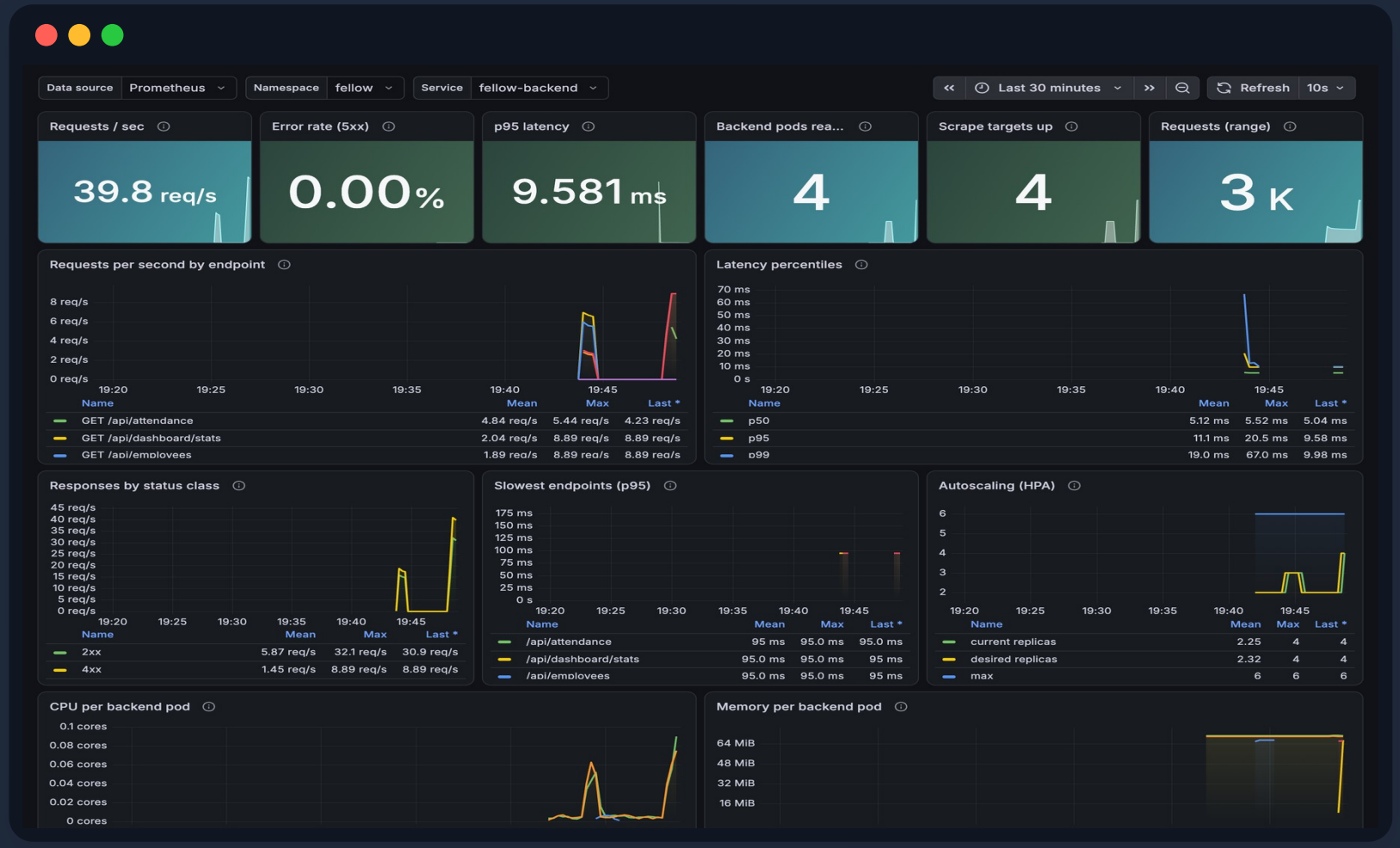


13
Grafana panels

~40
req / sec under load

~10 ms
p95 latency

UP
all scrape targets





When things broke

Trivy	3 HIGH CVEs in Starlette (CVE-2025-62727) and old Alpine libs. Upgraded, now 0 HIGH or CRITICAL.
Review	10 bugs found in a code review, each fixed with a regression test.
Race	Two replicas migrated at once. Fixed with a Postgres advisory lock.
Ingress	Controller landed on a node without port 80. Pinned to the control plane.
Labs	ImagePullBackOff and a Service with no endpoints, reproduced and fixed live.

KEY LEARNINGS

What I'd tell past me

Gate early

Tests and scans before images means nothing broken ever reaches the registry.

GitOps is calm

Git is the source of truth, so a deploy is just a reviewed commit.



"why is the pod pending?"

Measure first

Metrics showed the HPA working long before I guessed at limits.

Security is a step

Non-root images, scanned deps and no secrets in Git, every build.

Demo time

Push a commit, watch it go green, land on staging through Argo CD, then show up on the Grafana board.

Code	github.com/Manasvi-247/fellow-hrms
App	fellow.localhost
Metrics	grafana.localhost
GitOps	argocd.localhost



Thank you · Manasvi Sabbarwal